

Práctica 3: Técnicas de Búsqueda basadas en Trayectorias de Poblaciones para el Problema de Portfolio

Andrés Borja Duro

Curso académico 2025-2026

Índice

1. Descripción del problema	3
2. Aplicación de los algoritmos	4
2.1. Representación de las Soluciones	4
2.2. Retorno	4
2.3. Fórmulas	4
2.4. Función Objetivo	4
2.5. Operadores Comunes	4
2.5.1. Mutación	5
2.5.2. Método para inicializar posiciones del vector de vecinos	5
2.5.3. Método para inicializar índices de empresas	5
3. Descripción de los algoritmos	6
3.1. Búsqueda Multiarranque Básica (BMB)	6
3.2. Búsqueda Local Reiterada (ILS)	7
3.3. Enfriamiento Simulado (ES)	8
3.4. ILS-ES	10
3.5. Uso de Búsqueda Local (BL)	12
4. Estructura del código y manual de usuario	14
4.1. Estructura del código	14
4.2. Compilación	14
4.3. Ejecución	14
4.4. Gráficas	14
5. Experimentos y análisis de resultados	15
5.1. Descripción de los experimentos	15
5.2. Resultados	15
5.3. Análisis de resultados	17
6. Algoritmos extra	19
6.1. Extra 1: ILS-MOD	19
6.2. Extra 2: ES-MOD y ES-MOD2	19
7. Referencias	21

1. Descripción del problema

El problema del portfolio consiste en encontrar la asignación óptima de capital entre un conjunto de N activos financieros disponibles, de forma que se maximice el beneficio y se minimice el riesgo.

Cada solución del problema se representa mediante un vector de pesos $w = (w_1, w_2, \dots, w_N)$, donde w_i indica la proporción de capital invertida en el activo i . Deberán de cumplir ciertas restricciones que ahora veremos.

El beneficio asociado a una solución se calcula a partir de los retornos históricos de los activos, mientras que el riesgo se modela mediante la matriz de covarianzas, que recoge la dependencia entre los distintos activos. De esta forma, el problema plantea un compromiso entre maximizar la rentabilidad esperada y minimizar la variabilidad de dicha rentabilidad.

Este problema lo vamos a abordar mediante el uso de técnicas metaheurísticas.

2. Aplicación de los algoritmos

2.1. Representación de las Soluciones

Para representar una solución se usará un vector de pesos reales $w = (w_1, w_2, \dots, w_n)$ donde cada elemento w_i representa la proporción del capital total que se invertirá en la empresa i . Las restricciones para que una solución sea válida son las siguientes:

- **Suma de pesos igual a 1:** $\sum_{i=1}^n w_i \approx 1$. Se pone $\approx$ porque con valores reales las comparaciones se hacen con un cierto margen de error, como 10^{-8} .
- **Rango limitado:** $(w_i = 0) \vee (lo \leq w_i \leq hi), \forall i \in 1..N$. Los valores lo y hi se definirán más adelante según los datos con los que trabajemos.

2.2. Retorno

Para cada día t del periodo y en cada empresa i tenemos el retorno obtenido μ_{ti} que viene dado por:

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}}$$

Un valor positivo indica que ha habido mejora, y un valor negativo indica que se ha perdido dinero ese día.

2.3. Fórmulas

A partir de la matriz de retornos podremos calcular:

Concepto	Para qué se usa	Fórmula
Media por acción	Retorno medio de una empresa en el periodo T .	$\bar{\mu}_i = \left(\sum_{t=1}^T \mu_{ti} \right) / T$
Covarianza	Medir cómo varían conjuntamente dos acciones para evaluar su dependencia.	$\Sigma_{ij} = \frac{1}{T} \sum_{t=1}^T (\mu_{ti} - \bar{\mu}_i) \cdot (\mu_{tj} - \bar{\mu}_j)$
Beneficio	Calcular la ganancia total esperada de la cartera usando logaritmos.	$Beneficio(w) = \sum_i w_i \cdot \left(\sum_{t=1}^T \log(1 + \mu_{ti}) \right)$
Riesgo	Cuantificar la volatilidad o peligro de pérdida total de la cartera.	$Riesgo(w) = \sqrt{\sum_i \sum_j w_i \cdot \Sigma_{ij} \cdot w_j}$

Cuadro 1: Métricas

2.4. Función Objetivo

Función objetivo a maximizar: equilibra el beneficio obtenido penalizando el riesgo

$$fitness(w) = Beneficio(w) - \lambda \cdot Riesgo(w)$$

Donde usaremos un valor de $\lambda = 500$

El pseudocódigo es similar al de las prácticas anteriores.

2.5. Operadores Comunes

En cuanto a los operadores comunes seguimos usando algunos de los de las prácticas 1 y 2, y además añadimos algunos más también.

2.5.1. Mutación

Es la mutación que usamos tanto en ILS, como en ILS-ES. Consiste en barajar un vector con los índices de las empresas, y después tomamos un número de empresas que barajaremos entre ellas usando los índices barajados para que sea menos costoso.

```
1 funcion mutar(solucion, numero_empresas_barajar):
2     barajar(indices_empresas)
3
4     para i <- 0 hasta numero_empresas_barajar - 1 con paso 2 hacer
5         sol_aux <- solucion[indices_empresas[i]]
6         solucion[indices_empresas[i]] <- solucion[indices_empresas[i + 1]]
7         solucion[indices_empresas[i + 1]] <- sol_aux
8     fin para
9 fin funcion
```

2.5.2. Método para inicializar posiciones del vector de vecinos

Lo usamos en todos los algoritmos, ya que el vector de posiciones de los vecinos se usa en todos.

```
1 funcion inicializarPosiciones(tam_sol):
2     posiciones.clear()
3     posiciones.reserve(tam_sol * tam_sol)
4
5     para i <- 0 hasta tam_sol - 1 hacer
6         para j <- 0 hasta tam_sol - 1 hacer
7             posiciones.push_back({i, j})
8         fin para
9     fin para
10 fin funcion
```

2.5.3. Método para inicializar índices de empresas

Los usamos tanto en ILS, como en ILS-ES para luego usar los índices de empresas en la mutación

```
1 funcion inicializarIndicesEmpresas(tam):
2     indices_empresas.clear()
3     indices_empresas.reserve(tam)
4
5     para i <- 0 hasta tam - 1 hacer
6         indices_empresas.push_back(i)
7     fin para
8 fin funcion
```

3. Descripción de los algoritmos

3.1. Búsqueda Multiarranque Básica (BMB)

El algoritmo consistirá en inicializar 5 soluciones y aplicar búsqueda local sobre cada una de estas 5 soluciones con un máximo de 2000 evaluaciones para cada una de las 5. Usaremos $ratio = 0,4$ para los intercambios de la búsqueda local.

```
1  funcion optimize(problema, max_evals):
2      evaluaciones <- 0
3
4      tam_sol <- problema.obtenerTamanoSolucion()
5      rango <- problema.obtenerRangoDominio()
6      lo <- rango.inferior, hi <- rango.superior
7
8      // Inicializamos el vector con todas las parejas de posiciones para BL
9      inicializarPosiciones(tam_sol)
10     tam_pos <- tam_sol * tam_sol
11
12     // Instancia de clase Búsqueda Local
13     bl <- crearBusquedaLocalAuxiliar(lo, hi, RATIO, MAX_EVALS_BL, max_evals)
14
15     soluciones <- reservar(N_SOLUCIONES_INICIALES)
16     fitness_soluciones <- reservar(N_SOLUCIONES_INICIALES)
17
18     // Inicializacion soluciones iniciales y aplicacion de busqueda local
19     para i <- 0 hasta N_SOLUCIONES_INICIALES - 1 hacer
20         soluciones[i] <- crearSolucion(problema)
21         fitness_soluciones[i] <- fitness(soluciones[i])
22         evaluaciones <- evaluaciones + 1
23
24         evals_BL <- 0
25         bl.aplicarBL(problema, soluciones[i], fitness_soluciones[i], posiciones,
26                     tam_pos, evals_BL)
27         evaluaciones <- evaluaciones + evals_BL
28     fin para
29
30     indice_mejor <- seleccionaMejor(fitness_soluciones)
31
32     retornar crearResultadoMH(soluciones[indice_mejor], fitness_soluciones[indice_mejor], evaluaciones)
33 fin funcion
```

3.2. Búsqueda Local Reiterada (ILS)

El algoritmo consiste en generar una solución inicial aleatoria y aplicar el algoritmo de búsqueda local sobre ella. Una vez obtenida la solución tras aplicarle la búsqueda local, se realizará una mutación sobre la mejor entre la obtenida y el mejor global, volviendo a aplicar el algoritmo de búsqueda local sobre esta solución mutada. Esto lo repetiremos un total de 5 veces, donde en la búsqueda local se realizarán un máximo de 1999 evaluaciones (no 2000 para nunca pasarnos de 10000 porque tenemos que dar 5 vueltas y hay una solución inicial y una mutación por vuelta), y se usará $ratio = 0,4$ para los intercambios de la búsqueda local. Para las mutaciones lo que hacemos es tomar un 20% aleatorio de empresas y cambiarlas de posición entre ellas.

```
1  funcion optimize(problema, max_evals):
2      evaluaciones <- 0
3      mejor_fitness_global <- -infinito
4      mejor_global <- vacio
5
6      tam_sol <- problema.obtenerTamanioSolucion()
7      rango <- problema.obtenerRangoDominio()
8      lo <- rango.inferior, hi <- rango.superior
9      numero_empresas_barajar <- PORCENTAJE_MUTACION * tam_sol
10
11     // Inicializamos el vector con todas las parejas de posiciones para BL
12     inicializarPosiciones(tam_sol)
13     tam_pos <- tam_sol * tam_sol
14
15     indices_empresas <- reservar(tam_sol)
16     inicializarIndicesEmpresas(tam_sol)
17
18     // Instancia de clase Búsqueda Local
19     bl <- crearBusquedaLocalAuxiliar(lo, hi, RATIO, MAX_EVALS_BL, max_evals)
20
21     // Inicializamos con una solución aleatoria
22     mejor_global <- crearSolucion(problema)
23     mejor_fitness_global <- fitness(mejor_global)
24     evaluaciones <- evaluaciones + 1
25
26     // Aplicamos la primera BL
27     evals_BL <- 0
28     bl.aplicarBL(problema, mejor_global, mejor_fitness_global, posiciones, tam_pos,
29                 evals_BL)
30     evaluaciones <- evaluaciones + evals_BL
31
32     // Inicializacion poblacion mediante mutacion y busqueda local
33     para i <- 1 hasta N_VUELTAS - 1 hacer
34         // Mutamos la mejor solución
35         solucion_aux <- mejor_global
36         mutar(solucion_aux, tam_sol, numero_empresas_barajar)
37         fitness_aux <- fitness(solucion_aux)
38         evaluaciones <- evaluaciones + 1
39
40         evals_BL <- 0
41         bl.aplicarBL(problema, solucion_aux, fitness_aux, posiciones, tam_pos, evals_BL)
42         evaluaciones <- evaluaciones + evals_BL
43
44         // Actualizamos si hemos mejorado la global
45         si fitness_aux > mejor_fitness_global entonces
46             mejor_global <- solucion_aux
47             mejor_fitness_global <- fitness_aux
48         fin si
49     fin para
50
51     retornar crearResultadoMH(mejor_global, mejor_fitness_global, evaluaciones)
52 fin funcion
```

3.3. Enfriamiento Simulado (ES)

El enfriamiento simulado es un algoritmo inspirado en el calentamiento de metales realizado durante una forja. Es parecido a la búsqueda local, pero tenemos un parámetro llamado *temperatura* que puede permitir movernos a una solución peor en fases iniciales, y que va reduciéndose conforme avanza la ejecución, y permitiendo empeorar cada vez menos.

Usamos el esquema de Cauchy modificado para el enfriamiento: $T_{k+1} = \frac{T_k}{1+\beta \cdot T_k}$ con $\beta = \frac{T_0 - T_f}{M \cdot T_0 \cdot T_f}$ donde M es el número de enfriamientos a realizar, T_0 es la temperatura inicial y T_f es la temperatura final.

Para la temperatura inicial: $T_0 = \frac{\mu \cdot |\text{Fitness}(S_0)|}{-\ln(\varphi)}$ donde S_0 es una solución inicial y tomaremos $\varphi = 0,3$ y $\mu = 0,2$. La temperatura final será $T_f = 10^{-3}$.

Es importante comprobar siempre que la temperatura inicial sea mayor a la final. También destacar que no paramos cuando la inicial es menor que la temperatura final, pararemos cuando agotemos las evaluaciones o cuando no mejoremos.

Para el bucle interno se usará: $\text{max_vecinos} = 10m$ y $\text{max_exitos} = 1m$ siendo m el tamaño de la solución. El número máximo de evaluaciones será 10000. Por lo tanto, el número de enfriamientos $M = 10000/\text{max_vecinos}$.

```
1 function optimize(problema, max_evals):
2     evaluaciones <- 0
3     mejor_fitness_global <- -infinito
4     mejor_global <- vacio
5
6     tam_sol <- problema.obtenerTamanoSolucion()
7     rango <- problema.obtenerRangoDominio()
8     lo <- rango.inferior, hi <- rango.superior
9
10    // Inicializamos el vector de posiciones para solo tener que hacerlo una vez
11    inicializarPosiciones(tam_sol)
12    tam_pos <- tam_sol * tam_sol
13
14    // Inicializacion
15    solucion <- crearSolucion(problema)
16    fitness <- fitness(solucion)
17    mejor_global <- solucion
18    mejor_fitness_global <- fitness
19    evaluaciones <- evaluaciones + 1
20    T_inicial <- MU * absoluto(fitness) / (-logaritmo(PHI))
21
22    // La temperatura incial debe ser mayor a la final
23    mientras T_inicial <= T_FINAL hacer
24        solucion <- crearSolucion(problema)
25        fitness <- fitness(solucion)
26        mejor_global <- solucion
27        mejor_fitness_global <- fitness
28        evaluaciones <- evaluaciones + 1
29        T_inicial <- MU * absoluto(fitness) / (-logaritmo(PHI))
30    fin mientras
31
32    max_vecinos <- CONSTANTE_VECINOS * tam_sol
33    max_exitos <- CONSTANTE_EXITOS * tam_sol
34    M <- max_evals / max_vecinos // Numero de enfriamientos
35    hay_exito <- verdadero
36    beta <- (T_inicial - T_FINAL) / (M * T_inicial * T_FINAL * 1.0)
37    T <- T_inicial
38
39    // Condicion de parada
40    mientras evaluaciones < max_evals y hay_exito hacer
41        n_vecinos <- 0
42        n_exitos <- 0
43        barajar(posiciones)
44        pos <- 0
45
46        // Condicion enfriamiento
47        mientras n_vecinos < max_vecinos y n_exitos < max_exitos y evaluaciones <
48            max_evals y pos < tam_pos hacer
49            // Generamos el vecino
50            pos_i <- posiciones[pos].primero
```



```

50     pos_j <- posiciones[pos].segundo
51
52     si pos_i != pos_j entonces
53         valor_anterior_i <- solucion[pos_i]
54         valor_anterior_j <- solucion[pos_j]
55
56         si valor_anterior_i > lo y valor_anterior_j < hi entonces
57             n_vecinos <- n_vecinos + 1
58             tope_1 <- 1.0 - (lo / valor_anterior_i)
59             tope_2 <- (hi - valor_anterior_j) / valor_anterior_i
60             tope_3 <- RATIO
61
62             // Nos quedamos con el menor tope
63             ratio <- minimo({tope_1, tope_2, tope_3})
64
65             si ratio == 0 entonces
66                 pos <- pos + 1
67                 continuar
68             fin si
69
70             solucion[pos_i] <- (1 - ratio) * valor_anterior_i
71             solucion[pos_j] <- valor_anterior_j + ratio * valor_anterior_i
72
73             si problema.esValida(solucion) entonces
74                 new_fit <- fitness(solucion)
75                 evaluaciones <- evaluaciones + 1
76                 incremento_fitness <- fitness - new_fit
77                 u <- aleatorio(0.0, 1.0)
78
79                 si incremento_fitness < 0 o (u <= exponencial(-
                    incremento_fitness / (T * 1.0))) entonces
80                     fitness <- new_fit
81                     n_exitos <- n_exitos + 1
82                     barajar(posiciones)
83                     pos <- 0
84
85                     si fitness > mejor_fitness_global entonces
86                         mejor_global <- solucion
87                         mejor_fitness_global <- fitness
88                     fin si
89                 sino
90                     // Recuperamos la solucion como era porque el cambio o no
91                     // es valido o no mejora
92                     solucion[pos_i] <- valor_anterior_i
93                     solucion[pos_j] <- valor_anterior_j
94                     pos <- pos + 1
95                 fin si
96             sino
97                 solucion[pos_i] <- valor_anterior_i
98                 solucion[pos_j] <- valor_anterior_j
99                 pos <- pos + 1
100             fin si
101         sino
102             pos <- pos + 1
103         fin si
104     sino
105         pos <- pos + 1
106     fin si
107     fin mientras
108
109     T <- T / (1.0 + beta * T)
110     hay_exito <- n_exitos > 0
111     retornar crearResultadoMH(mejor_global, mejor_fitness_global, evaluaciones)
112 fin funcion

```

3.4. ILS-ES

Es similar al ILS con búsqueda local pero en vez de aplicar búsqueda local aplicamos enfriamiento simulado. Lo repetiremos 5 veces al igual que el anterior, y además todo lo relativo a ES es similar a lo explicado en ese punto, pero cambiando *max_evals* por *MAX_EVALS_ES* que será 1999. (Para nunca pasarnos de 10000 porque tenemos que dar 5 vueltas y hay una solución inicial y una mutación por vuelta).

```
1 funcion optimize(problema, max_evals):
2     evaluaciones <- 0
3     mejor_fitness_global <- -infinito
4     mejor_global <- vacio
5     tam_sol <- problema.obtenerTamanoSolucion()
6     rango <- problema.obtenerRangoDominio()
7     lo <- rango.inferior, hi <- rango.superior
8     numero_empresas_barajar <- PORCENTAJE_MUTACION * tam_sol
9
10    // Inicializamos el vector de posiciones y los indices
11    inicializarPosiciones(tam_sol)
12    tam_pos <- tam_sol * tam_sol
13    indices_empresas <- reservar(tam_sol)
14    inicializarIndicesEmpresas(tam_sol)
15
16    max_vecinos <- CONSTANTE_VECINOS * tam_sol
17    max_exitos <- CONSTANTE_EXITOS * tam_sol
18    M <- MAX_EVALS_ES / max_vecinos // Numero de enfriamientos por iteracion de ILS
19
20    // Bucle principal de la Búsqueda Local Iterada (ILS)
21    para i <- 0 hasta N_VUELTAS - 1 hacer
22        fitness <- 0.0
23        solucion <- vacio
24        si i == 0 entonces
25            // Inicializamos con una solucion aleatoria
26            mejor_global <- crearSolucion(problema)
27            mejor_fitness_global <- fitness(mejor_global)
28            fitness <- mejor_fitness_global
29            solucion <- mejor_global
30        sino
31            // Mutamos el mejor
32            solucion <- mejor_global
33            mutar(solucion, numero_empresas_barajar)
34            fitness <- fitness(solucion)
35        fin si
36        evaluaciones <- evaluaciones + 1
37        hay_exito <- verdadero
38        T_inicial <- MU * absoluto(fitness) / (-logaritmo(PHI))
39        // La temperatura inicial debe ser mayor a la final, mutamos hasta conseguirlo
40        mientras T_inicial <= T_FINAL hacer
41            solucion <- mejor_global
42            mutar(solucion, numero_empresas_barajar)
43            fitness <- fitness(solucion)
44            evaluaciones <- evaluaciones + 1
45
46            si fitness > mejor_fitness_global entonces
47                mejor_global <- solucion
48                mejor_fitness_global <- fitness
49            fin si
50            T_inicial <- MU * absoluto(fitness) / (-logaritmo(PHI))
51        fin mientras
52
53        beta <- (T_inicial - T_FINAL) / (M * T_inicial * T_FINAL * 1.0)
54        T <- T_inicial
55        evaluaciones_ES <- 0
56
57        // Fase de Enfriamiento Simulado (Local Search del ILS)
58        mientras evaluaciones_ES < MAX_EVALS_ES y hay_exito hacer
59            n_vecinos <- 0
60            n_exitos <- 0
61            barajar(posiciones)
```

```

62     pos <- 0
63     // Condicion de enfriamiento
64     mientras n_vecinos < max_vecinos y n_exitos < max_exitos y evaluaciones_ES
        < MAX_EVALS_ES y pos < tam_pos hacer
65         pos_i <- posiciones[pos].primero
66         pos_j <- posiciones[pos].segundo
67         si pos_i != pos_j entonces
68             valor_anterior_i <- solucion[pos_i]
69             valor_anterior_j <- solucion[pos_j]
70             si valor_anterior_i > lo y valor_anterior_j < hi entonces
71                 n_vecinos <- n_vecinos + 1
72                 tope_1 <- 1.0 - (lo / valor_anterior_i)
73                 tope_2 <- (hi - valor_anterior_j) / valor_anterior_i
74                 tope_3 <- RATIO
75                 ratio <- minimo({tope_1, tope_2, tope_3})
76                 si ratio == 0 entonces
77                     pos <- pos + 1
78                     continuar
79             fin si
80             solucion[pos_i] <- (1 - ratio) * valor_anterior_i
81             solucion[pos_j] <- valor_anterior_j + ratio * valor_anterior_i
82             si problema.esValida(solucion) entonces
83                 new_fit <- fitness(solucion)
84                 evaluaciones_ES <- evaluaciones_ES + 1
85                 incremento_fitness <- fitness - new_fit
86                 u <- aleatorio(0.0, 1.0)
87                 si incremento_fitness < 0 o (u <= exponencial(-
                    incremento_fitness / T)) entonces
88                     fitness <- new_fit
89                     n_exitos <- n_exitos + 1
90                     barajar(posiciones)
91                     pos <- 0
92                     si fitness > mejor_fitness_global entonces
93                         mejor_global <- solucion
94                         mejor_fitness_global <- fitness
95                     fin si
96                 sino
97                     // Revertir cambios
98                     solucion[pos_i] <- valor_anterior_i
99                     solucion[pos_j] <- valor_anterior_j
100                    pos <- pos + 1
101                fin si
102            sino
103                solucion[pos_i] <- valor_anterior_i
104                solucion[pos_j] <- valor_anterior_j
105                pos <- pos + 1
106            fin si
107        sino
108            pos <- pos + 1
109        fin si
110    sino
111        pos <- pos + 1
112    fin si
113    fin mientras
114    T <- T / (1.0 + beta * T)
115    hay_exito <- n_exitos > 0
116    fin mientras
117    // Sumamos las evaluaciones invertidas en esta vuelta de ES
118    evaluaciones <- evaluaciones + evaluaciones_ES
119    // Actualizamos si el resultado final del ES ha mejorado la global
120    si fitness > mejor_fitness_global entonces
121        mejor_global <- solucion
122        mejor_fitness_global <- fitness
123    fin si
124    fin para
125    retornar crearResultadoMH(mejor_global, mejor_fitness_global, evaluaciones)
126 fin funcion

```

3.5. Uso de Búsqueda Local (BL)

En esta práctica usaremos la Búsqueda Local en BMB e ILS, y el código será común a los dos. Se mantiene igual que en la práctica anterior, pero ahora con $ratio = 0,4$. El pseudocódigo se muestra en la siguiente página.

```

1  funcion aplicarBL(problem, sol, fit, posiciones, tam_pos, evaluaciones):
2      barajar(posiciones)
3      pos <- 0
4      maximo_evaluaciones_BL <- minimo(evaluaciones + max_evals_BL, max_evals_global)
5      mientras pos < tam_pos y evaluaciones < maximo_evaluaciones_BL hacer
6          pos_i <- posiciones[pos].first
7          pos_j <- posiciones[pos].second
8          si pos_i != pos_j entonces
9              valor_anterior_i <- sol[pos_i]
10             valor_anterior_j <- sol[pos_j]
11             si valor_anterior_i > lo y valor_anterior_j < hi entonces
12                 tope_1 <- 1.0 - (lo / valor_anterior_i)
13                 tope_2 <- (hi - valor_anterior_j) / valor_anterior_i
14                 tope_3 <- ratio_max
15                 // Nos quedamos con el menor tope y ese es lo maximo que podemos
16                 intercambiar
17                 ratio <- min({tope_1, tope_2, tope_3})
18                 si ratio == 0 entonces
19                     pos <- pos + 1
20                     continuar
21             fin si
22             sol[pos_i] <- (1 - ratio) * valor_anterior_i
23             sol[pos_j] <- valor_anterior_j + ratio * valor_anterior_i
24
25             si problem.isValid(sol) entonces // hay que hacerlo porque por
26                 redondeo a veces se quedan < lo
27                 new_fit <- problem.fitness(sol)
28                 evaluaciones <- evaluaciones + 1
29                 si new_fit > fit entonces // Encontrado el primero mejor
30                     fit <- new_fit
31                     Random.shuffle(posiciones)
32                     pos <- 0
33                 sino // Recuperamos la solucion como era porque el cambio o no es
34                     valido o no mejora
35                     sol[pos_i] <- valor_anterior_i
36                     sol[pos_j] <- valor_anterior_j
37                     pos <- pos + 1
38                 fin si
39             sino
40                 sol[pos_i] <- valor_anterior_i
41                 sol[pos_j] <- valor_anterior_j
42                 pos <- pos + 1
43             fin si
44         sino
45             pos <- pos + 1
46         fin si
47     fin mientras
48 fin funcion

```

4. Estructura del código y manual de usuario

4.1. Estructura del código

Esta es la estructura general del proyecto, donde en la carpeta build están los archivos generados automáticamente durante la compilación del proyecto, en configuraciones están las configuraciones de cada algoritmo, en datos-portfolio-2526 los csv con los datos de las bolsas, en inc los archivos .h y en src los .cpp, en common los archivos comunes a todos, y en resultados las gráficas y el script para generarlas.

```
|-- build
|-- CMakeLists.txt
|-- common
|-- configuraciones
|-- datos_portfolio_2526
|-- ejecutar_todo.sh
|-- inc
|-- resultados
|-- LICENSE
|-- main.cpp
|-- README.md
|-- src
```

4.2. Compilación

Nos situamos en la carpeta software y hacemos lo siguiente:

- cd build
- rm -rf *
- cmake ..
- make
- cd ..

4.3. Ejecución

Ahora después de hacer la compilación anterior nos situamos en la carpeta software y hacemos:

- ./ejecutar_todo.sh semilla_inicial
- Ejemplo de uso ./ejecutar_todo.sh 1 empezará con la semilla 1 y terminará con 50 la última ejecución

4.4. Gráficas

Nos situamos en la carpeta software de nuevo:

- cd resultados
- python3 graficas.py

5. Experimentos y análisis de resultados

5.1. Descripción de los experimentos

Se utilizan datos bursátiles reales (2015-2025) de empresas con presencia continua en las siguientes bolsas de valores:

- `ibex_35_data.csv`: IBEX 35 con 30 empresas y 2813 días. Parámetros ($lo = 0,005, hi = 0,08$).
- `syp_100_data.csv`: S&P 100 con 97 empresas y 2765 días. Parámetros ($lo = 0,005, hi = 0,05$).
- `syp_500_data.csv`: S&P 500 con 457 empresas y 2765 días. Parámetros ($lo = 0,005, hi = 0,02$).

Para que la comparación sea justa se utilizará un máximo de 10000 evaluaciones, una evaluación es cada vez que llamamos a la función *fitness*

- $maxevals = 10000$

Se va a utilizar como semilla desde 1, sumando 1 en cada iteración hasta llegar a 50 incluido.

5.2. Resultados

Resultados medios de los algoritmos en cada bolsa:

Algoritmo	Fitness (2015–2024)	Fitness (2025)	Beneficio (2025)	Evaluaciones	Tiempo (s)
Greedy	-4.5584	-3.2545	0.0401	1.0000	0.0005
RS	-5.0948	-3.3365	0.0490	10000.0000	0.0257
BL	-4.5794	-3.1105	0.0214	1201.5600	0.0026
AM-BEST	-4.5687	-3.1191	0.0275	10002.0200	0.0344
BMB	-4.5640	-3.1076	0.0216	6087.1000	0.0134
ILS	-4.5790	-3.1083	0.0224	5039.2800	0.0096
ES	-4.5984	-3.1155	0.0210	9964.2200	0.0260
ILS-ES	-4.5935	-3.1152	0.0214	10000.0000	0.0271

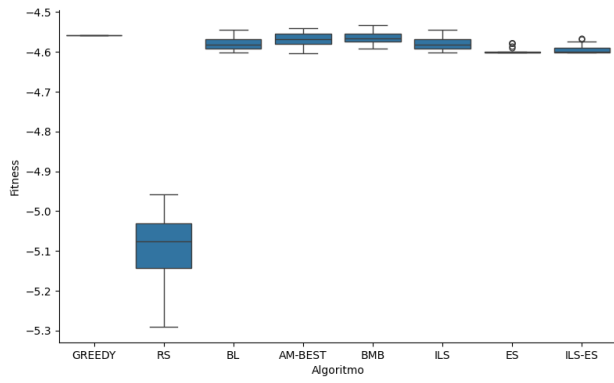
Cuadro 2: Ibex35 resultados (medias)

Algoritmo	Fitness (2015–2024)	Fitness (2025)	Beneficio (2025)	Evaluaciones	Tiempo (s)
Greedy	-2.8350	-3.6912	0.0929	1.0000	0.0013
RS	-3.4369	-4.1258	0.0784	10000.0000	0.1009
BL	-3.0298	-3.8724	0.0978	5229.4600	0.0800
AM-BEST	-2.9092	-3.7428	0.0950	10000.0000	0.2607
BMB	-2.9585	-3.8425	0.0947	10000.0000	0.2060
ILS	-3.0298	-3.8747	0.0951	9999.8000	0.1552
ES	-3.1301	-3.9086	0.0947	10000.0000	0.2838
ILS-ES	-3.1017	-3.8959	0.0957	10000.0000	0.3779

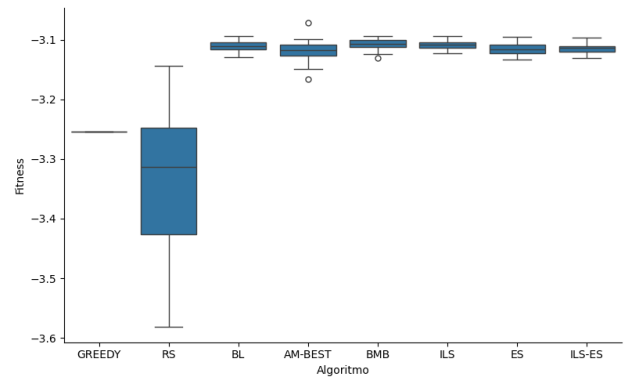
Cuadro 3: SyP100 resultados (medias)

Algoritmo	Fitness (2015–2024)	Fitness (2025)	Beneficio (2025)	Evaluaciones	Tiempo (s)
Greedy	-2.7495	-3.7768	0.0134	1.0000	0.0082
RS	-3.7173	-4.4645	0.0144	10000.0000	0.4938
BL	-3.2789	-4.2016	0.0074	10000.0000	0.9878
AM-BEST	-2.8946	-3.9627	0.0153	10000.0000	3.6628
BMB	-3.1761	-4.1084	0.0159	10000.0000	2.5953
ILS	-3.3049	-4.1859	0.0135	10000.0000	1.3401
ES	-3.7786	-4.5796	0.0052	10000.0000	3.5437
ILS-ES	-3.7637	-4.5158	0.0084	10000.0000	5.2382

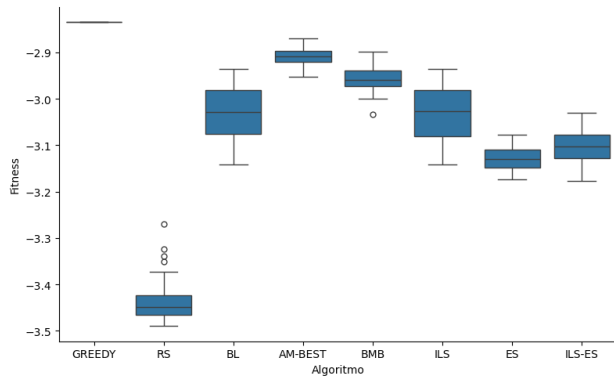
Cuadro 4: Resultados (medias)



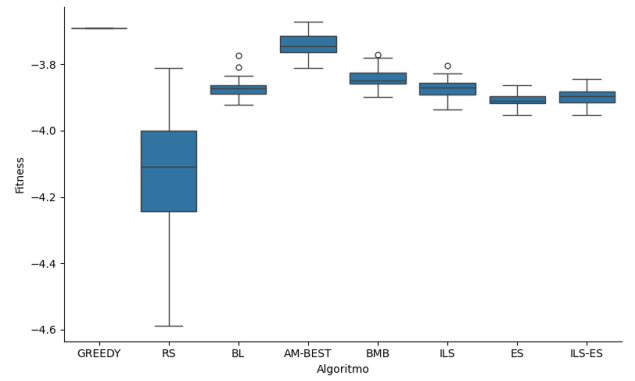
(a) IBEX35 2015-2024



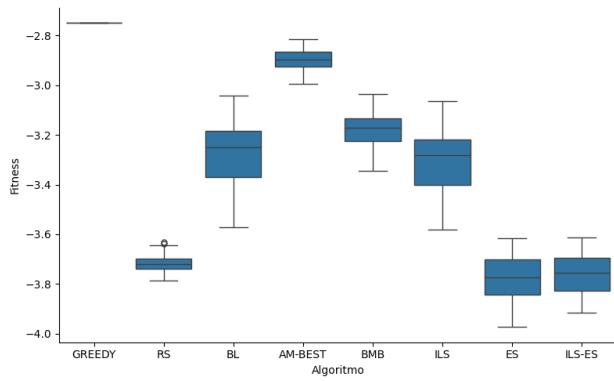
(b) IBEX35 2025



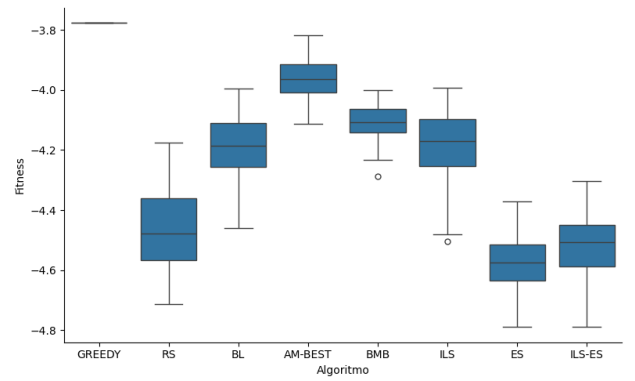
(c) SyP100 2015-2024



(d) SyP100 2025



(e) SyP500 2015-2024



(f) SyP500 2025

Figura 1: Conjunto de resultados

5.3. Análisis de resultados

Para poder hacer un análisis mejor, primero vamos a mostrar como se ordenan los algoritmos de menor a mayor ranking medio. La media de rankings de cada algoritmo según su fitness histórico:

Cuadro 5: Ranking y ranking medio de los algoritmos en cada dataset.

Algoritmo	IBEX35	SyP 100	SyP 500	Ranking Medio
Greedy	1.0	1.0	1.0	1.00
AM-BEST	3.0	2.0	2.0	2.33
BMB	2.0	3.0	3.0	2.67
ILS	4.0	4.5	5.0	4.5
BL	5.0	4.5	4.0	4.5
ILS-ES	6.0	6.0	7.0	6.33
ES	7.0	7.0	8.0	7.33
RS	8.0	8.0	6.0	7.33

El peor de todos los algoritmos de esta práctica claramente es Enfriamiento Simulado. Si que es verdad que con tamaños pequeños de soluciones, como en el Ibex35 no hay tanta diferencia, pero con soluciones más grandes es incluso peor que la búsqueda aleatoria, algo que sorprende. Esto se debe a que con soluciones grandes, la temperatura la enfriará muy pocas veces, en el caso concreto del SyP500 si tenemos mas o menos 500 empresas, el número de enfriamientos se calcula $M = \frac{10000}{\max \vec{vecinos}} = \frac{10000}{10 \cdot 500} = 2$, aunque también por el número de éxitos del bucle interno, podemos enfriar alguna vez más, lo he estado comprobando y como mucho enfría 4 o 5 veces por ejecución, lo que hace que se acepten muchísimas soluciones malas al principio, y esa aceptación no disminuya mucho conforme avanza el algoritmo. En el otro extremo, con el Ibex35, si que puede enfriar muchas veces lo que le permite funcionar correctamente, y no obtiene resultados tan alejados de los otros algoritmos de la práctica.

El algoritmo ILS obtiene mejores resultados que ILS-ES. Ambos se comportan de forma similar con soluciones pequeñas, algo que tiene sentido ya que la búsqueda local puede explorar todos los vecinos de la solución en cada iteración de ILS, en Ibex35 hay $30 \times 30 = 900$ vecinos, por lo que con 2000 evaluaciones evaluamos todos los vecinos de sobra, esto se refleja en el numero medio de evaluaciones, que nos sobran, ya que solo hace 5039 de media. En el caso de ILS-ES le permite enfriar en ES muchas veces como hemos comentado anteriormente. El problema viene conforme aumenta el tamaño de la solución, en el SyP500, en la búsqueda local ya hay $457 \times 457 = 208.849$ vecinos, por lo que le es imposible explorar todos con 2000 evaluaciones. En ILS-ES se obtienen en SyP500 peores resultados que en la búsqueda aleatoria, esto se debe a lo comentado anteriormente respecto al número de enfriamientos, que en cada vuelta será igual a 1 o 2 como mucho.

Por otro lado puede parecer que la mutación es demasiado conservadora, ya que BMB con soluciones nuevas obtiene mejores resultados que ILS en todos los datasets. Pero el problema realmente viene del propio algoritmo ILS, da igual que hagamos una mutación en la que intercambiamos todas las empresas, porque hay dos problemas. El primer problema es que el intercambio mantiene los pesos, por lo tanto tampoco cambiamos tanto la solución como si hiciéramos una nueva, como BMB. El segundo problema, que es el principal, es que cuando hacemos una mutación y lanzamos búsqueda local si no mejoramos al mejor global que teníamos, entonces damos un paso atrás y volvemos a mutar el mejor global y hacerle búsqueda local, por lo que esas evaluaciones se desperdician. Para demostrar que da igual que intercambiamos todas las empresas lo he añadido como algoritmo extra comparándolo con BMB e ILS. Por todo lo dicho anteriormente y como ILS-ES es peor que ILS, pues entonces es peor que BMB.

De los algoritmos nuevos implementados en esta práctica el mejor es BMB. Es muy bueno en el Ibex35 ya que como el tamaño de la solución es muy pequeño, esto le permite explorar todo el vecindario con la búsqueda local y salir antes de gastar todas las evaluaciones. Como se puede ver en la tabla, en el Ibex35 solo gasta 6087 evaluaciones de media, por lo que le sobran bastantes. En el SyP100 y SyP500 sigue siendo bueno en comparación a los implementados en la práctica. Comparándolo con la búsqueda local obtiene resultados muy similares, claramente en el Ibex35 tiene que ser mejor BMB, porque BL recorre el vecindario entero, ya que solo realiza 1200 evaluaciones, luego BMB es como hacer BL 5 veces. Sin embargo en SyP100 y SyP500 ya no podemos recorrer el vecindario entero en cada búsqueda local interna de BMB por lo que aquí no está tan claro como en el Ibex35. BMB es mejor en estos también porque al trabajar con vecindarios enormes, tanto en BL como en BMB solo recorreremos para cada solución inicial una parte ínfima del vecindario (aunque se recorra más parte la BL, no es significativa la diferencia), sin embargo en BMB partimos de 5 soluciones aleatorias que están más repartidas, lo que permite tener más diversidad y explorar más regiones, lo que hace obtener mejores soluciones que BL.

El algoritmo AM-BEST obtiene mejores resultados que los algoritmos de esta práctica. BMB, o ILS (si no mejoramos) olvidan la información del problema en cada reinicio. Sin embargo, AM-BEST, mediante el operador de cruce, logra combinar soluciones buenas dando saltos poco a poco de forma más inteligente en el espacio de búsqueda, algo que los algoritmos de trayectoria no pueden replicar. Además, al aplicar la búsqueda local sobre los mejores (que suelen ser los más prometedores) hace una combinación muy buena entre exploración y explotación. En los algoritmos nuevos lo que más falla es la exploración como se ha visto y no tanto la explotación.

En cuanto a los boxplots como se ha visto en prácticas anteriores conforme aumenta el tamaño de la solución, se puede ver que tienen más altura los boxplots, es decir, se distribuyen de forma más amplia los resultados. Como son algoritmos de trayectoria única, podemos ver que respecto a la práctica anterior si lo comparamos con AM-Best, como dependemos mucho de las soluciones iniciales, tienen mayor altura que AM-Best, que es el mejor de la anterior.

Los tiempos de ejecución, todos los algoritmos tienen que mezclar el vector del vecindario que ya hemos comentado en otras prácticas, donde en el SyP500 al ser enorme hace que aumente significativamente el tiempo de ejecución. Por tanto entre los cuatro tardan más cuantas más veces en la búsqueda local interna o en ES, barajan el vector de vecinos comentado. Se puede ver que ES e ILS-ES son los que más tardan, porque al aceptar soluciones peores por el problema de que enfrían pocas veces, como ya se ha comentado antes, aceptan muchas soluciones peores, por lo que tienen que barajarlo cada vez que la aceptan.

En la siguiente página muestro algunos algoritmos extra.

6. Algoritmos extra

6.1. Extra 1: ILS-MOD

Este algoritmo lo hago para explicar que el problema en ILS respecto a BMB como he explicado en las conclusiones, no es que sea demasiado conservador el intercambio de empresas. Para ello muestro los resultados de ILS-MOD que lo único que cambia es que se intercambian el 90 % de las empresas en la mutación. Los resultados obtenidos son los siguientes y confirman lo que queríamos demostrar, que no se acerca más a BMB:

Algoritmo	Fitness (2015–2024)	Evaluaciones	Tiempo (s)		
BMB	-4.5640	-3.1076	0.0216	6087.1000	0.0134
ILS	-4.5790	-3.1083	0.0224	5039.2800	0.0096
ILS-MOD	-4.5780	-3.1101	0.0213	5983.0400	0.0135

Cuadro 6: Ibex35 resultados (medias)

Algoritmo	Fitness (2015–2024)	Evaluaciones	Tiempo (s)		
BMB	-2.9585	-3.8425	0.0947	10000.0000	0.2231
ILS	-3.0298	-3.8747	0.0951	9999.8000	0.1767
ILS-MOD	-3.0305	-3.8725	0.0936	10000.0000	0.2366

Cuadro 7: SyP100 resultados (medias)

Algoritmo	Fitness (2015–2024)	Evaluaciones	Tiempo (s)		
BMB	-3.1761	-4.1084	0.0159	10000.0000	3.3840
ILS	-3.3049	-4.1859	0.0135	10000.0000	1.5810
ILS-MOD	-3.3106	-4.1755	0.0149	10000.0000	2.2201

Cuadro 8: SyP500 Resultados (medias)

6.2. Extra 2: ES-MOD y ES-MOD2

La primera modificación consiste en que cuando llegemos a una temperatura baja, es decir, que sea inferior a la final, con la idea de explorar más, reiniciemos desde una mutación de esta solución. Los resultados obtenidos al final son muy similares y no hay mejora apenas.

La otra modificación consiste en definir un número máximo de veces que podemos empeorar, para hacerlo más restrictivo todavía y conseguir mejorar un poco en SyP500 los problemas comentados en las conclusiones. Este número varía conforme se enfría la temperatura. Empezamos con $max_peores = tam_sol/2$ y cada vez que enfriamos $max_peores = max(1, max_peores/2)$.

Algoritmo	Fitness (2015–2024)	Evaluaciones	Tiempo (s)		
ES	-4.5984	-3.1155	0.0210	9964.2200	0.0262
ES-MOD	-4.5986	-3.1164	0.0215	10000.6600	0.0317
ES-MOD2	-4.5907	-3.1130	0.0205	7109.9000	0.0152

Cuadro 9: Ibex35 resultados (medias)

Algoritmo	Fitness (2015–2024)	Evaluaciones	Tiempo (s)		
ES	-3.1301	-3.9086	0.0947	10000.0000	0.3229
ES-MOD	-3.1338	-3.8994	0.0947	10000.8000	0.6120
ES-MOD2	-3.0873	-3.8967	0.0964	9415.7800	0.1637

Cuadro 10: SyP100 resultados (medias)

Algoritmo	Fitness (2015-2024)	Evaluaciones	Tiempo (s)		
ES	-3.7786	-4.5796	0.0052	10000.0000	3.7579
ES-MOD	-3.7901	-4.5587	0.0068	10000.7000	8.6188
ES-MOD2	-3.6125	-4.4731	0.0030	10000.0000	1.6978

Cuadro 11: SyP500 Resultados (medias)

7. Referencias

1. Guión de las prácticas y material de teoría de la asignatura.
2. OpenAI. (2026). *ChatGPT* (Versión GPT-4o) <https://chatgpt.com>
3. Google. (2026). *Gemini* (Versión 1.5 Pro) <https://gemini.google.com>